Assignment 6: Retrieval-Augmented Generation (RAG) - Machine Learning Knowledge Assistant

You will build an **end-to-end Retrieval-Augmented Generation (RAG) application** using a well-known machine learning book such as Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow in PDF format.

Your goal is to develop a system that can:

1. Retrieve relevant content from the book
2. Generate accurate answers to technical questions related to Machine Learning and Deep Learning

Dataset Description

The dataset for this assignment is a **PDF version of a machine learning book**.

Content Description

Component	Description
Book PDF	Contains structured knowledge on ML & DL concepts
Chapters	Organized sections such as supervised learning, neural networks, etc.
Text Content	Source for retrieval and answer generation

Part 1 — Data Understanding & Preprocessing

Perform document loading and preprocessing.

Tasks:

1. Load the PDF document
2. Extract text from all pages
3. Explore document structure:
   a. Number of pages
   b. Text quality issues etc.
4. Split text into chunks

Suggested approach:

- Chunk size: 500–1000 tokens or more based on experimentations
- Overlap: 50–150 tokens or more based on experimentations

Part 2 — Embedding & Vector Database

Build the retrieval foundation of the RAG system.

Tasks:

1. Generate embeddings for all text chunks
2. Store embeddings in a vector database
3. Implement similarity search mechanism

Recommended tools:

- Embeddings: OpenAI / Gemini / SentenceTransformers
- Vector DB: FAISS / ChromaDB

Deliverables:

- Description of embedding model used
- Explanation of vector storage approach
- Example of retrieved chunks for a sample query

Part 3 — Retrieval Pipeline

Implement query-based retrieval.

Tasks:

1. Accept user query
2. Convert query into embedding
3. Retrieve top-k relevant chunks
4. Experiment with different values of k (e.g., 3, 5, 7)

Part 4 — Answer Generation (RAG)

Integrate retrieval with an LLM.

Tasks:

1. Pass retrieved chunks as context to LLM

2. Design prompt to ensure grounded responses
3. Generate answers using the LLM
4. Ensure:
    a. Answers are based only on retrieved content
    b. Minimal hallucination

Deliverables:

- Prompt design explanation
- Sample queries and generated answers
- Evidence of grounding (context-based answers)

Part 5 — End-to-End RAG Application

Build a working Q&A system.

Tasks:

1. Integrate:
    a. Document processing
    b. Retrieval system
    c. LLM generation
2. Allow real-time question answering

Deliverables:

- Functional RAG application
- Demonstration of multiple queries